Reporte Técnico de Actividades Práctico-Experimentales Nro. 07

1. Datos de Identificación del Estudiante y la Práctica

Nombre del estudiante	Cael Alejandro Soto Castillo
Asignatura	Simulación
Ciclo	5
Unidad	U2: Generación de números pseudoaleatorios y generación de variables aleatorias
Resultado de aprendizaje de la unidad	R2. Interpreta los resultados de una Simulación, bajo los principios de solidaridad, transparencia, responsabilidad y honestidad.
Práctica Nro.	07
Título de la Práctica	Estimación de resultados mediante el método de Monte Carlo
Nombre del Docente	José O. Guamán Q.
Fecha	12 de noviembre de 2026 13 de noviembre de 2026
Horario	Jueves 10h30 a 11h30 Viernes 07h30 a 09h30
Lugar	Aula
Tiempo planificado en el Sílabo	3

2. Objetivo(s) de la Práctica

Aplicar el método de Monte Carlo para obtener aproximaciones numéricas mediante experimentos aleatorios.

1. Implementar un experimento Monte Carlo.
2. Analizar la influencia del tamaño de muestra sobre el error.
3. Comparar resultados obtenidos en Python y Java.

3. Materiales y reactivos

- Computador.
- Lenguaje de programación Python.
- Lenguaje de programación Java.
- Entorno de desarrollo integrado.
- Generador de números pseudoaleatorios.

4. Equipos y herramientas

- Python.
- NumPy.

- Matplotlib.
- Pandas.
- JDK.
- IntelliJ IDEA, Eclipse o NetBeans.
- Visual Studio Code.

5. Procedimiento / Metodología Ejecutada

5.1. Fundamento teórico

El método de Monte Carlo utiliza muestras aleatorias para aproximar resultados. Para estimar π, se generan puntos en un cuadrado y se determina cuáles pertenecen a un círculo inscrito. La idea se parece a lanzar dardos al azar a un cuadrado que tiene un círculo dibujado dentro: como el círculo ocupa una parte conocida del cuadrado ($\pi/4$), la proporción de dardos que caen dentro permite aproximar π.

La aproximación de π se expresa mediante:

$$\pi \approx 4 \cdot \left(\frac{N_c}{N} \right)$$

donde N_c es el número de puntos dentro del círculo y N es el número total de puntos generados. La diferencia entre la estimación y el valor real se mide con el error absoluto:

$$\text{Error absoluto} = |\pi_{\text{estimado}} - \pi|$$

Como las dos pruebas que usé para comparar resultados son de Chi-cuadrado (χ^2), también las resumo aquí. La prueba compara lo observado (O) con lo esperado (E) y, si el estadístico χ^2 resulta grande, significa que lo observado se aleja de lo esperado:

$$\chi^2 = \sum \frac{(O - E)^2}{E}$$

Se contrasta la hipótesis nula H_0 (no hay diferencia) con un nivel de significancia $\alpha = 0{,}05$. Con 1 grado de libertad, el valor crítico es 3,8415: si χ^2 es menor que ese valor, o si el valor p es mayor que 0,05, no se rechaza H_0.

5.2. Implementación computacional

Estos fueron los pasos que seguí para realizar la práctica:

1. **Definí el experimento.** Usé un cuadrado de lado 2 (de -1 a 1) con un círculo de radio 1 en el centro. Cada punto se genera con dos números aleatorios entre -1 y 1 y está dentro del círculo si cumple $x^2 + y^2 \leq 1$.
2. **Organicé el programa en modelo, vista y controlador** en Python y en Java. El modelo contiene los generadores, el experimento y las pruebas; la vista imprime los resultados, guarda los CSV y dibuja las gráficas; y el controlador sigue los pasos de la ficha.
3. **Usé dos generadores de números aleatorios.** El primero es un generador congruencial lineal (LCG) propio, programado igual en los dos lenguajes ($a = 1664525$, $c = 1013904223$, $m = 2^{32}$ y semilla 12345), por lo que produce los mismos números en Python y en Java. El segundo es el generador que trae cada lenguaje (`random` en Python y `java.util.Random` en Java), también con la semilla 12345. Los agregué para poder comparar y responder por qué los resultados pueden diferir.

4. **Ejecuté el experimento** con los cuatro tamaños de muestra de la ficha, una vez por tamaño y por generador. En cada caso calculé π, el error absoluto y el tiempo de ejecución.
5. **Repetí el experimento 10 veces por tamaño** con semillas distintas (12345, 12346, ...) usando el LCG propio. Lo hice porque una sola ejecución puede tener buena o mala suerte, y así pude ver con más claridad cómo cambia el error.
6. **Apliqué las pruebas Chi-cuadrado.** La de bondad de ajuste compara los puntos dentro del círculo con los esperados si π fuera el valor real, y la de homogeneidad compara los resultados de Python con los de Java.
7. **Hice las gráficas** de los puntos, la estimación, el error, la convergencia, los tiempos y los estadísticos Chi-cuadrado (Tabla 1).

El proyecto se ejecuta desde la terminal con un solo comando (`./ejecutar.sh`), que compila y ejecuta Java y después Python.

5.3. Tamaños de muestra

La ficha establece ejecutar el experimento con los siguientes tamaños de muestra:

- 1 000 puntos.
- 10 000 puntos.
- 100 000 puntos.
- 1 000 000 de puntos.

5.4. Variables registradas

Para cada tamaño de muestra registré las siguientes variables:

- Estimación de π.
- Error absoluto.
- Tiempo de ejecución en Python.
- Tiempo de ejecución en Java.

5.5. Implementación cruzada

Primero ejecuté el experimento en Java, que guarda sus resultados en archivos CSV. Después, el programa de Python hace el mismo experimento, lee esos archivos y compara lo que obtuvo cada lenguaje: los puntos dentro del círculo, la estimación de π, el estadístico χ^2 y los tiempos. En Java hice un calentamiento antes de medir los tiempos, porque la primera vez que se ejecuta un bucle es más lenta y habría falseado el resultado.

5.6. Gráficas del informe

Tabla 1: Figuras del informe, con el nombre del archivo de cada una.

Fig.	Archivo (en `figures/`)	Contenido
1	`puntos.png`	Puntos lanzados al cuadrado, dentro y fuera del círculo, para cada tamaño.
2	`estimacion_pi.png`	Estimación de π en cada tamaño, con los dos generadores y los dos lenguajes.
3	`error_vs_n.png`	Error absoluto según el tamaño de muestra (escala logarítmica).
4	`convergencia.png`	Estimación de π a lo largo de una ejecución de 1 000 000 de puntos.
5	`tiempos.png`	Tiempo de ejecución de Python y Java por tamaño.
6	`chi_cuadrado.png`	Estadísticos χ^2 contra el valor crítico.

6. Resultados esperados

Obtener dos implementaciones funcionales que produzcan la misma secuencia cuando utilizan los mismos parámetros. El estudiante debe armar una tabla con los datos generados por ambos programas e ir comparando; además, debe dar sus conclusiones.

La Tabla 2 sigue el formato de la ficha y se llenó con los resultados del generador congruencial propio, que es el que produce la misma secuencia en los dos lenguajes. Los tiempos están en milisegundos.

Tabla 2: Resultados del experimento Monte Carlo con el LCG propio (formato de la ficha).

N	π estimado	Error	Tiempo Python (ms)	Tiempo Java (ms)
1 000	3,188000	0,046407	0,37	0,07
10 000	3,151600	0,010007	3,85	0,75
100 000	3,133240	0,008353	37,25	5,62
1 000 000	3,141948	0,000355	394,39	15,53

6.1. Aspectos para el análisis de los resultados

El análisis deberá considerar los siguientes puntos establecidos en la ficha:

1. ¿Cómo cambia la estimación de π cuando aumenta el número de puntos?
2. ¿Qué relación existe entre el tamaño de muestra y el error absoluto?
3. ¿En cuál de los tamaños de muestra se obtuvo una mejor aproximación de π?
4. ¿Qué lenguaje presentó menor tiempo de ejecución?
5. ¿Por qué los resultados obtenidos en Python y Java pueden ser diferentes?
6. ¿Qué importancia tiene la generación de números pseudoaleatorios en el método Monte Carlo?
7. ¿El aumento del número de puntos garantiza que cada ejecución obtenga un resultado más cercano al valor real?
8. ¿Qué aplicaciones reales puede tener el método Monte Carlo en ingeniería, economía, informática y ciencias?

7. Resultados

7.1. Estimación de π

Con cada generador ejecuté el experimento una vez por tamaño de muestra. Con el LCG propio, Python y Java obtuvieron exactamente los mismos puntos dentro del círculo (N_c) y, por tanto, la misma estimación (Tabla 3). Con el generador de cada lenguaje, en cambio, los resultados fueron distintos (Tabla 4).

Tabla 3: Resultados con el LCG propio (iguales en Python y en Java).

N	N_c	π estimado	Error absoluto	Valor p (χ^2)
1 000	797	3,188000	0,046407	0,3715
10 000	7 879	3,151600	0,010007	0,5423
100 000	78 331	3,133240	0,008353	0,1077
1 000 000	785 487	3,141948	0,000355	0,8287

Tabla 4: Resultados con el generador que trae cada lenguaje (`random` en Python y `java.util.Random` en Java).

N	π Python	π Java	Error Python	Error Java	p Python	p Java
1 000	3,140000	3,232000	0,001593	0,090407	0,9755	0,0817
10 000	3,133200	3,148000	0,008393	0,006407	0,6093	0,6964
100 000	3,137800	3,141360	0,003793	0,000233	0,4652	0,9643
1 000 000	3,143076	3,142416	0,001483	0,000823	0,3664	0,6161

Las gráficas de las Figuras 1 y 2 muestran los puntos lanzados y cómo cambia la estimación con el tamaño.

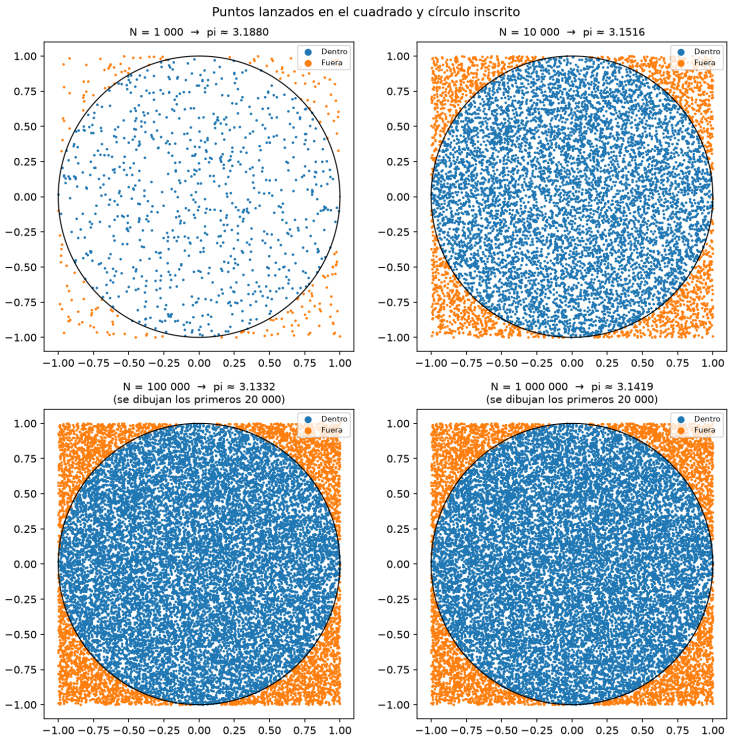


Figura 1: Puntos lanzados al cuadrado con el LCG propio: los azules caen dentro del círculo y los naranjas fuera. Para 100 000 y 1 000 000 solo se dibujan los primeros 20 000.

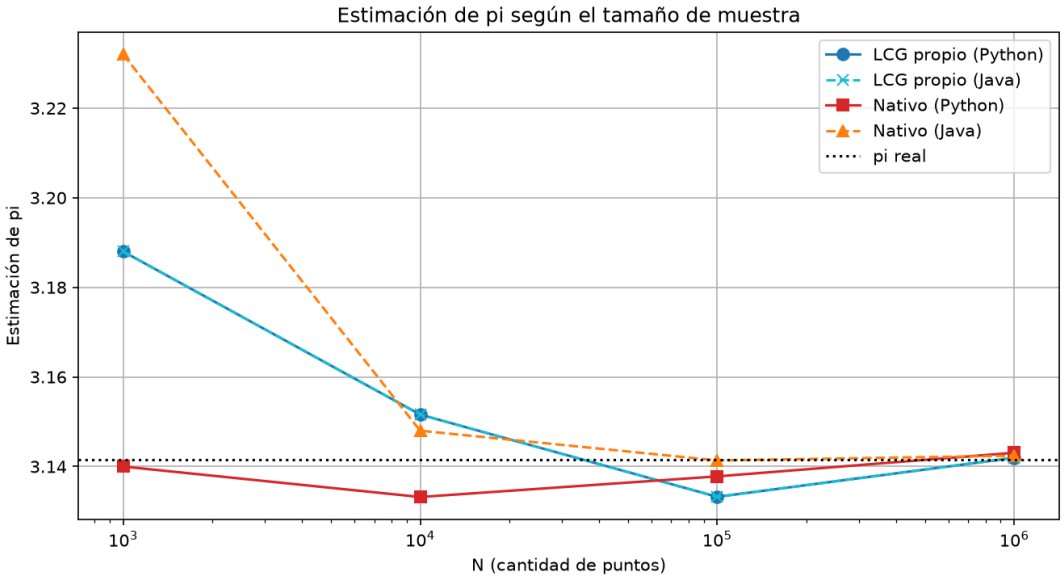


Figura 2: Estimación de π según el tamaño de muestra. Las líneas del LCG propio en Python y en Java coinciden; las del generador de cada lenguaje no.

7.2. Error absoluto

Como una sola ejecución puede tener buena o mala suerte, repetí el experimento 10 veces por tamaño con el LCG propio y semillas distintas. La Tabla 5 compara el error promedio observado con el error que se espera teóricamente, que baja en proporción a $1/\sqrt{N}$.

Tabla 5: Error absoluto en 10 repeticiones por tamaño (LCG propio) y error medio teórico.

N	Error medio observado	Error mínimo	Error máximo	Error medio teórico
1 000	0,03928	0,00641	0,10159	0,04143
10 000	0,00692	0,00001	0,02241	0,01310
100 000	0,00370	0,00009	0,00835	0,00414
1 000 000	0,00120	0,00036	0,00365	0,00131

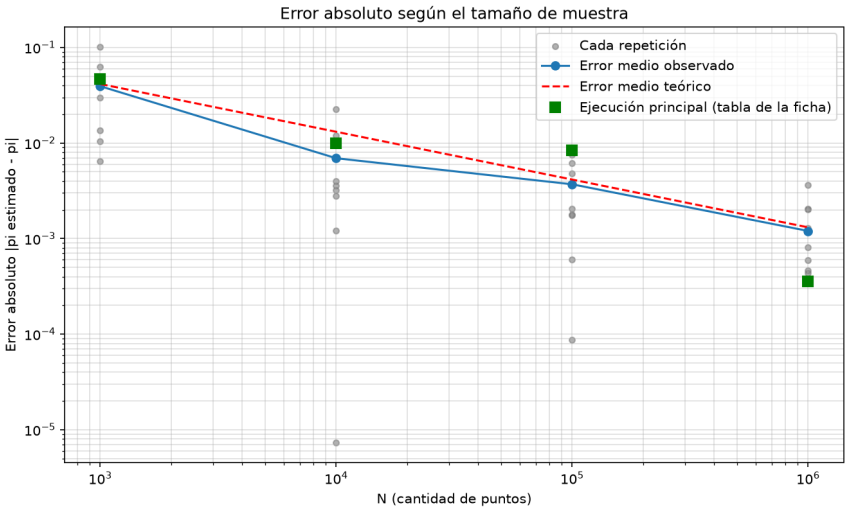


Figura 3: Error absoluto según el tamaño de muestra. Los puntos grises son las repeticiones, la línea azul su promedio y la línea roja el error teórico.

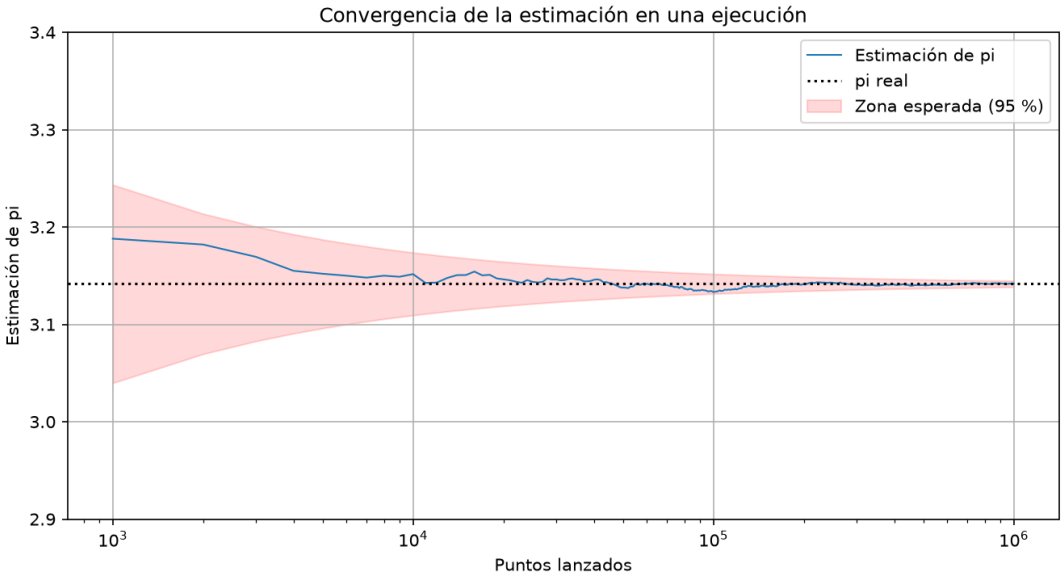


Figura 4: Estimación de π en una ejecución de 1 000 000 de puntos. La zona roja es el rango donde se espera que esté la estimación el 95 % de las veces.

7.3. Tiempo de ejecución

En la Tabla 6 están los tiempos que registré en cada lenguaje, en milisegundos. Estos valores dependen del computador y cambian un poco en cada ejecución.

Tabla 6: Tiempo de ejecución en milisegundos según el lenguaje, el generador y el tamaño de muestra.

N	LCG: Python	LCG: Java	Nativo: Python	Nativo: Java
1 000	0,37	0,07	0,19	0,04
10 000	3,85	0,75	1,83	0,40
100 000	37,25	5,62	18,77	4,22
1 000 000	394,39	15,53	201,71	42,44

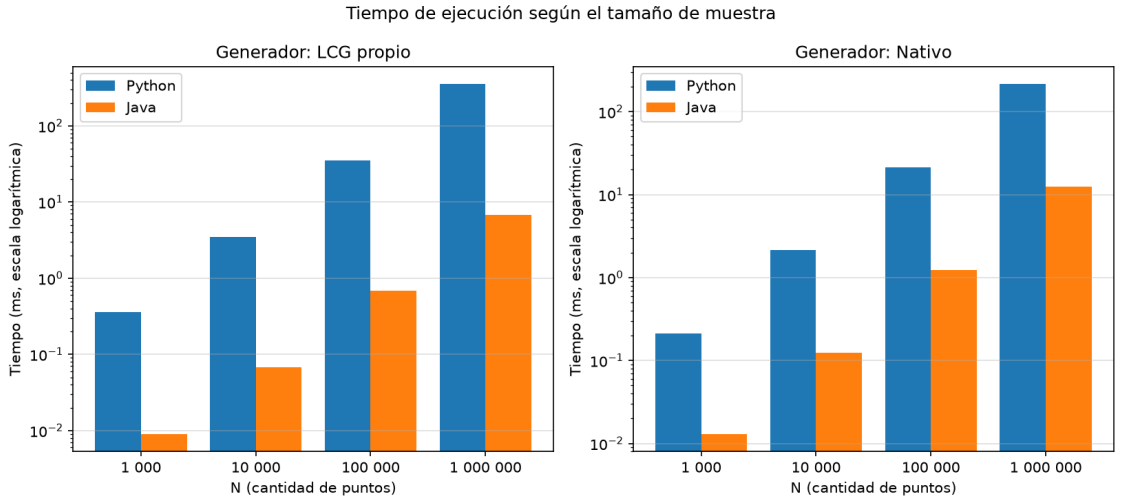


Figura 5: Tiempo de ejecución de Python y Java según el tamaño de muestra (escala logarítmica).

7.4. Comparación entre Python y Java

Para comparar los dos lenguajes con un cuadro estadístico, usé la prueba Chi-cuadrado de homogeneidad. Con una tabla de 2×2 (puntos dentro y fuera del círculo en Python y en Java) se contrasta la hipótesis nula de que los dos lenguajes obtienen la misma proporción de puntos dentro. Con el LCG propio los resultados son idénticos, así que $\chi^2 = 0$ en todos los tamaños. La Tabla 7 muestra el caso del generador de cada lenguaje.

Tabla 7: Chi-cuadrado de homogeneidad entre Python y Java con el generador de cada lenguaje (1 grado de libertad, $\alpha = 0{,}05$, χ^2 crítico $= 3{,}8415$).

N	χ^2	Valor p	Decisión
1 000	1,6318	0,2014	No se rechaza H_0
10 000	0,4058	0,5241	No se rechaza H_0
100 000	0,2346	0,6281	No se rechaza H_0
1 000 000	0,0808	0,7762	No se rechaza H_0

También apliqué la prueba de bondad de ajuste, que compara los puntos dentro del círculo con los esperados si π fuera el valor real. Sus valores p aparecen en las Tablas 3 y 4: todos son mayores que 0,05.

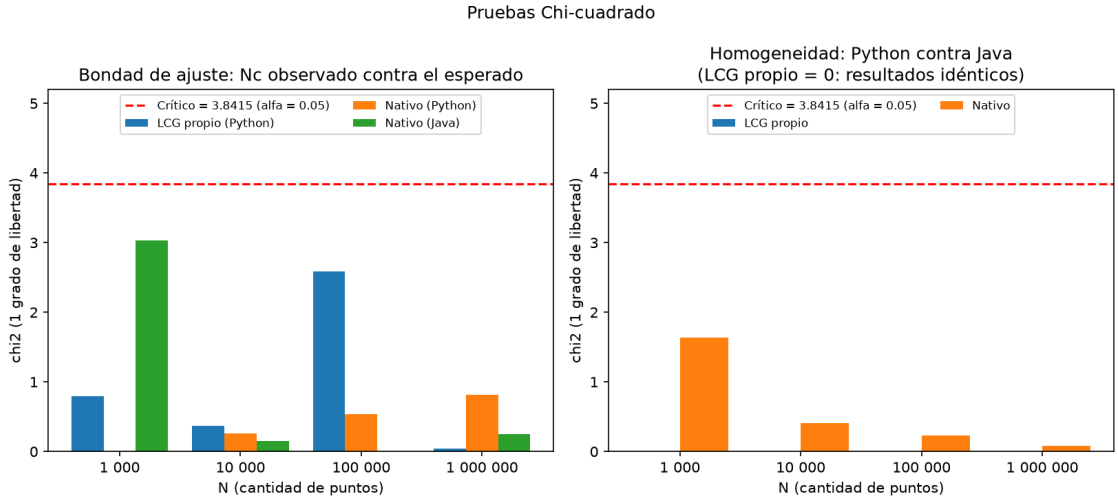


Figura 6: Estadístico χ^2 de la bondad de ajuste (izquierda) y de la homogeneidad entre Python y Java (derecha). La línea roja es el valor crítico.

8. Análisis de Resultados

Para analizar los resultados seguí los puntos que plantea la ficha.

8.1. Cambio de la estimación al aumentar los puntos

Con pocos puntos la estimación se aleja bastante de π: con 1 000 puntos obtuve 3,188 (Tabla 3). A medida que aumenté el tamaño de muestra, la estimación se fue acercando, hasta llegar a 3,141948 con 1 000 000 de puntos. En la Figura 4 se ve esto de forma clara: al principio la línea fluctúa mucho y después se estabiliza alrededor del valor real. También noté que no es una bajada constante, porque la estimación sube y baja en el camino.

8.2. Relación entre el tamaño de muestra y el error absoluto

En general, a mayor tamaño de muestra, menor error. Con las 10 repeticiones (Tabla 5), el error promedio pasó de 0,039 con 1 000 puntos a 0,0012 con 1 000 000. Esto concuerda con el error teórico, que baja en proporción a $1/\sqrt{N}$: si el número de puntos se multiplica por 100, el error se divide más o menos entre 10. En la Figura 3 los puntos grises (las repeticiones) quedan cerca de la línea del error teórico, aunque varían bastante, sobre todo con pocos puntos.

8.3. Mejor aproximación de π

Con el LCG propio, la mejor aproximación fue con 1 000 000 de puntos (error de 0,000355). Pero con el generador de Java el mejor resultado fue con 100 000 puntos (error de 0,000233), mejor que el de 1 000 000 (0,000823). Esto me hizo ver que, aunque lo más probable es que más puntos den mejores resultados, en una sola ejecución puede haber suerte con una muestra más pequeña.

8.4. Lenguaje con menor tiempo de ejecución

Java fue más rápido que Python en todos los tamaños y con los dos generadores (Tabla 6). Por ejemplo, con el LCG propio y 1 000 000 de puntos, Java tardó 15,5 ms y Python 394 ms, es decir, Python tardó unas 25 veces más. Con el generador de cada lenguaje la diferencia fue menor, de unas 5 veces. Además, en los dos lenguajes el tiempo crece más o menos en proporción a la cantidad de puntos: si se multiplica N por 10, el tiempo también se multiplica por casi 10.

8.5. Por qué los resultados de Python y Java pueden ser diferentes

Con el LCG propio los dos lenguajes dieron lo mismo porque usan la misma fórmula, los mismos parámetros y la misma semilla, así que generan exactamente los mismos números. En cambio, con el generador que trae cada lenguaje los resultados fueron distintos (por ejemplo, 3,140 en Python y 3,232 en Java con 1 000 puntos), porque cada uno usa un algoritmo diferente (Python usa Mersenne Twister y Java un generador congruencial de 48 bits), así que aunque la semilla sea la misma, los números no coinciden. La prueba de homogeneidad (Tabla 7) mostró que esas diferencias no son significativas: en ningún tamaño se rechazó la hipótesis nula, es decir, la diferencia entre los dos es compatible con el azar.

8.6. Importancia de los números pseudoaleatorios

El método de Monte Carlo depende de que los puntos se repartan de forma pareja en el cuadrado. Si los números generados no fueran uniformes, la proporción de puntos dentro del círculo no sería $\pi/4$ y la estimación saldría mal. Además, como son pseudoaleatorios, con la misma semilla se puede repetir el experimento y obtener los mismos resultados, lo que me permitió comparar Python y Java.

8.7. Si más puntos garantizan un resultado más cercano en cada ejecución

No lo garantiza. Lo que mejora es el error promedio, no el resultado de cada ejecución. En mis resultados, con el generador de Python, el error con 10 000 puntos (0,008393) fue mayor que con 1 000 (0,001593), y con el generador de Java el error con 1 000 000 de puntos (0,000823) fue mayor que con 100 000 (0,000233). En la Tabla 5 también se ve que el error mínimo con 10 000 puntos (0,00001) es menor que el máximo con 1 000 000 (0,00365).

8.8. Aplicaciones reales del método de Monte Carlo

El método sirve cuando es difícil calcular un resultado de forma exacta. En ingeniería se usa, por ejemplo, para estimar la confiabilidad de sistemas; en economía, para evaluar riesgos financieros; en informática, para simulaciones y para probar algoritmos; y en las ciencias, para estudiar fenómenos de física o de química donde intervienen muchas variables.

9. Preguntas de Control

1. **¿Qué es Monte Carlo?**

   El método de Monte Carlo es una técnica de simulación estocástica que utiliza números pseudoaleatorios para resolver problemas matemáticos, estudiar variables aleatorias y aproximar valores esperados. Permite obtener soluciones numéricas mediante el muestreo repetido y el cálculo de promedios, incluso en problemas de alta dimensión [1]–[3].

2. **¿Por qué el error disminuye al aumentar las muestras?**

   Según la Ley de los Grandes Números, al incrementar el número de muestras independientes, la media muestral tiende a aproximarse al valor esperado. Además, la varianza del estimador disminuye según $\mathrm{Var}(\bar{X}) = \sigma^2/N$, por lo que el error estándar se reduce aproximadamente en proporción a $1/\sqrt{N}$. Sin embargo, esto no garantiza que cada nueva ejecución produzca una estimación más cercana al valor real [1], [3], [4].

3. **¿Qué relación existe entre precisión y costo computacional?**

   Existe una relación entre la precisión y el costo computacional: para reducir el error estándar a la mitad se necesita aproximadamente cuadruplicar el número de muestras. Por tanto, obtener estimaciones más precisas requiere más iteraciones y, generalmente, mayor tiempo de procesamiento [1], [4].

4. **¿Qué diferencias se observaron entre Python y Java?**

   En mi práctica, con el generador congruencial propio los dos lenguajes dieron exactamente los mismos resultados, y con el generador que trae cada lenguaje los resultados fueron distintos. Además, Java fue más rápido que Python en todos los tamaños de muestra. Fuera de lo que observé, las fuentes consultadas no presentan una comparación directa de rendimiento entre Python y Java. Sin embargo, señalan que los métodos de Monte Carlo pueden implementarse en distintos lenguajes de programación. Por ello, ambos permiten desarrollar este tipo de simulaciones, aunque los tiempos de ejecución pueden variar según la implementación y las herramientas utilizadas [1], [5].

5. **¿En qué otros problemas puede utilizarse Monte Carlo?**

   El método de Monte Carlo se aplica en diferentes áreas, como la evaluación de riesgos financieros, la optimización de inventarios, la integración numérica y la inferencia bayesiana. También permite estudiar eventos poco frecuentes, analizar la confiabilidad de redes y modelar fenómenos de física estadística [1], [2], [6].

10. Conclusiones

- Con esta práctica pude aplicar el método de Monte Carlo para aproximar π con un experimento sencillo de puntos al azar, y vi cómo un problema matemático se puede resolver con una simulación que se programa en pocas líneas.
- Al analizar el tamaño de muestra comprobé que, en promedio, más puntos reducen el error, pero que esa mejora es lenta y no es segura en cada ejecución. Hubo casos en

los que una muestra más pequeña dio un resultado mejor que una más grande.

- Al comparar Python y Java vi que, si los dos usan el mismo generador de números, obtienen los mismos resultados, y que si cada uno usa el suyo, los números cambian aunque estadísticamente se comportan igual. En los tiempos, Java fue claramente más rápido que Python.
- Usar pruebas Chi-cuadrado me ayudó a comparar los resultados con un criterio estadístico y no solo mirando los números: en ningún caso la diferencia fue significativa.

11. Declaración de uso de Inteligencia Artificial

En esta práctica utilicé dos herramientas de inteligencia artificial. Con **Claude** recibí ayuda para entender las fórmulas y el planteamiento del experimento para armar la estructura del proyecto en modelo, vista y controlador. Con **NotebookLM** obtuve las respuestas de las preguntas de control con sus referencias.

11.1. Prompts utilizados

Prompt 1: Claude (contextualización).

> *Entiende la siguiente ficha relacionada a Monte Carlo, dame una contextualización del método y ayudame armando una estructura (solo nombre de las clases no código) siguiendo modelo vista controlador para hacer las clases y empezar a hacer y saber que lógica usar. [Se adjuntó la ficha de la práctica 07.]*

Prompt 2: NotebookLM (preguntas de control).

> *Con las fuentes de este cuaderno responde las cinco preguntas de control: (1) ¿Qué es Monte Carlo?, (2) ¿Por qué el error disminuye al aumentar las muestras?, (3) ¿Qué relación existe entre precisión y costo computacional?, (4) ¿Qué diferencias existen entre Python y Java al implementar este tipo de métodos?, (5) ¿En qué otros problemas puede utilizarse Monte Carlo? Quiero respuestas directas y en un solo párrafo cada una, en lenguaje claro para un estudiante. Usa solo lo que dicen mis fuentes, si alguna no trae la información, dilo en lugar de inventarla. Entrégame todo en un archivo .md y, debajo de cada respuesta, pon las referencias que usaste en formato IEEE.*

11.2. Justificación del uso

Herramientas utilizadas.

- **Claude:** explicación de las fórmulas, estructura modelo-vista-controlador y primera versión del código, de las gráficas y del borrador del informe.
- **NotebookLM:** respuestas de las preguntas de control con sus referencias.

Trabajo realizado por el estudiante.

- El estudiante realizó el armado de las clases con ayuda de claude code y elaboró el documento mediante los resultados obtenidos en las ejecuciones (Java y Python) y las repsuestas a las preguntas de control con NotebookLM entregado por el docente.

12. Bibliografía

[1] J. S. Smith and D. T. Sturrock, *Simio and Simulation: Modeling, Analysis, Applications*. Sewickley, PA, USA: Simio LLC, 7th ed., August 2024.

[2] R. Salomone, L. F. South, C. Drovandi, D. P. Kroese, and A. M. Johansen, “Unbiased and consistent nested sampling via sequential Monte Carlo,” *Journal of the Royal Statistical Society Series B: Statistical Methodology*, vol. 87, pp. 1221–1238, May 2025.

[3] D. P. Kroese and Z. I. Botev, *An Advanced Course in Probability and Stochastic Processes*. August 2025. Course textbook manuscript.

[4] J. R. Baxter, “Introduction to probability,” February 2024. University of Minnesota. Available under Creative Commons Attribution-ShareAlike 4.0 International License.

[5] A. Heather, T. Monks, A. Harper, F. Alidoost, R. Challen, T. Slater, and N. Mustafee, “Reproducible analytical pipelines for healthcare discrete-event simulation: An open guide and worked examples,” *NIHR Open Research*, vol. 6, pp. 1–19, June 2026.

[6] S. Moka, C. Hirsch, V. Schmidt, and D. P. Kroese, “Efficient rare-event simulation for random geometric graphs via importance sampling,” 2024. Preprint / Research manuscript.

13. Anexos

13.1. Repositorio del proyecto

El código completo del proyecto (Python y Java) está en el repositorio del portafolio: https://github.com/C-ael/Simulacion_Portafolio.

Vídeo: https://drive.google.com/file/d/1C5pOpnfDDGlyRhe0v7hVDsjFQy4HZbMn/view?usp=sharing.